

(continued)

The wildcard type is an artifact of Scala's interoperability with Java's type system. Fortunately, when you have an `STRef[_ , Int]`, using it will cause a type error:

```
scala> new RunnableST[Int] {
  |   def apply[R] = for { x <- ref.read } yield x }
error   : type mismatch;
found   : ST[_$1,Int]
required: ST[R,Int]
          def apply[R] = for { x <- ref.read } yield x }
          ^
```

This type error is caused by the fact that the wildcard type in `ref` represents some concrete type that only `ref` knows about. In this case it's the `S` type that was bound in the `apply` method of the `RunnableST` where it was created. Scala is unable to prove that this is the same type as `R`. Therefore, even though it's possible to abuse the wildcard type to get the naked `STRef` out, this is still safe since we can't use it to mutate or access the state.

14.2.4 Mutable arrays

Mutable references on their own aren't all that useful. Mutable arrays are a much more compelling use case for the `ST` monad. In this section, we'll define an algebra for manipulating mutable arrays in the `ST` monad and then write an in-place quicksort algorithm compositionally. We'll need primitive combinators to allocate, read, and write mutable arrays.

Listing 14.3 An array class for our `ST` monad

Scala requires an implicit `Manifest` for constructing arrays.

```
sealed abstract class STArray[S,A](implicit manifest: Manifest[A]) {
  protected def value: Array[A]
  def size: ST[S,Int] = ST(value.size)

  def write(i: Int, a: A): ST[S,Unit] = new ST[S,Unit] {
    def run(s: S) = {
      value(i) = a
      ((), s)
    }
  }

  def read(i: Int): ST[S,A] = ST(value(i))

  def freeze: ST[S,List[A]] = ST(value.toList)

  object STArray {
    def apply[S,A:Manifest](sz: Int, v: A): ST[S, STArray[S,A]] =
      new STArray[S,A] {
        lazy val value = Array.fill(sz)(v)
      }
  }
}
```

Write a value at the given index of the array.

Read the value at the given index of the array.

Turn the array into an immutable list.

Construct an array of the given size filled with the value `v`.